

Tower Climber — Build Brief for Replit Agent

Hand this file to Replit Agent alongside: the Godot starter project zip, [tower-climber-progression-model.xlsx](#), and (if possible) 3-4 short gameplay clips of the reference game.

Build instruction for the agent: You are in **Plan & Verify mode**. Do not generate code, build application structure, or assemble the game until Section 0.5's verification step is complete. Once verification is done: import and extend the attached Godot 4 starter project — don't start a new project from scratch. The starter already has the core loop (tap-to-spawn, auto-attack, floor scaling, collapsible drawer UI) working. Build each milestone below on top of it, one at a time, and confirm each one runs before moving to the next.

0. Engineering Ground Rules (apply to every section below)

- **Stats and formulas are data, never hardcoded numbers in logic.** Extend the [TUNABLE ASSUMPTIONS](#) pattern already in [Main.gd](#) for every new system (gear, skills, soldiers) — one visible, editable place for every number, never a magic number typed inline into game logic.
- **Use Godot's [Resource](#) class for item/skill/soldier data**, not just script-level constants. Write one blueprint script (e.g. [ItemData.gd](#)) with empty typed fields (id, display_name, stat_affected, family_id, bonus_per_level...), then create one small [.tres](#) Resource file per actual item. Balancing a stat means editing a Resource file's numbers, never touching the code that uses it.
- **Centralize shared values in an Autoload singleton ([Globals.gd](#))** once more than one script needs the same constant — don't let the same number live in two different scripts' local const blocks, since that's exactly the "100 edits" problem this whole approach is meant to prevent.
- **Prefer components over duplicated logic.** A reusable [HealthComponent](#) or [HitboxComponent](#) node that any entity (hero, soldier, monster) can use, rather than writing near-identical health/combat logic inside each entity's own script.
- **One generic formula, reused per instance — don't hand-code near-duplicate logic per item/skill.** E.g. a single "stat-boost item" function instantiated once per gear piece, not copy-pasted with small tweaks each time.
- **IDs for logic, Display Names for UI — see Section 11.** Never let game logic, save data, or formula matching reference a display string.
- **Plan for big numbers now.** Stats will reach the thousands/millions/billions range. Build a number-formatting utility (K/M/B/T style display) early, and flag if/when a proper big-number math approach is needed rather than hitting a silent precision wall later.

- **Object pooling for frequently spawned/destroyed things** (heroes, soldiers, monsters) rather than naive instantiate/destroy every time — matters for mobile performance given how often these spawn.
- **Structure data with save/load in mind from the start**, even before save/load is actually built — if IDs are already the natural save key, persistence is a small addition later instead of a retrofit.
- **Commit to git after each working milestone**, not just at the end — gives real rollback points for a tuning-heavy project like this.
- **Sanity-check formulas against the spreadsheet**. A few debug assertions early (e.g. "floor 100 enemy HP should equal the value the spreadsheet computes") catch drift between the validated math and the implementation cheaply.
- **Placeholder art/audio first, always** — get the loop fun with default shapes/sounds before any visual polish pass, per the starter kit README's suggested order.

0.5. Plan & Verify Workflow — Do This Before Any Building

1. I will provide reference screenshots of the source game (not video — static images only), each paired with a short written note on anything the screenshot alone can't show: animation timing, hit-feedback duration, transition speed, pacing. Review these first. Do not write code yet.
2. I will then provide this brief and the progression spreadsheet as my own documentation.
3. Compare the two: cross-examine what the screenshots/notes show against what this brief specifies.
4. Produce a **Verification Report** before any building begins, structured as:
 - **Verified:** what the brief already gets right against the visual evidence
 - **Discrepancies:** anywhere the brief's numbers/mechanics conflict with what the screenshots show
 - **Missing elements:** mechanics, transitions, or UI states visible in the screenshots but not covered in the brief
 - **Proposed adjustments:** specific changes needed to reconcile the two
5. **Important caveat:** several formulas in this brief and the spreadsheet are reverse-engineered from community data, not confirmed source values, and are already known to be approximate in at least one case (a gear-leveling formula that didn't fully match a real data point at high levels — see the spreadsheet's Gear Stacking Model notes). Treat conflicts as genuine open questions to flag, not as the brief automatically being right.
6. Only after this report is reviewed and any adjustments agreed on should building begin, starting with the milestone order in Section 10's handoff checklist.

0.6. Communication Standard — I Am Not a Programmer

Every script you generate must include a short **plain-English translation immediately before the code block**, stating:

- What the script does, in plain terms
- Where its stats/data come from (confirm it's a Resource or `Globals.gd` reference, not a hardcoded value)
- Confirmation that there are zero magic numbers hardcoded into any math

If asked to "audit" a script, re-check it against Section 0's rules, rewrite anything that violates them, and state plainly what was fixed. Treat this audit request as a standing tool I can invoke on anything that looks off — a raw number inside a formula, a string comparison like `item.name == "..."` instead of an ID/type check, or a duplicated near-identical script where one generic version should exist instead.

1. Core Loop

- Single persistent "tower" screen — always visible, never navigates away
- Tapping the tower screen spawns a hero (bounded by the summon cap — see Section 3)
- Heroes auto-attack the current floor's monster continuously; no manual targeting
- When a floor's monster dies: award gold, advance to the next floor, next monster scales up per the Floor Scaling formula (see Section 5)
- Below the tower, a collapsible drawer with four tabs — **Heroes & Skills, Soldiers, Weapons, Shop** — stays accessible without leaving the tower view
- A static-but-rotating banner ad sits at the very top of the screen — never covers gameplay, changes roughly once a minute

2. Visual & Audio Feel

- 2D top-down / vertical tower framing, not side-scrolling
- Minimal, uncluttered UI — gold count, floor number, and the drawer handle are the only persistent HUD elements
- A single looping SNES-style background track — simple, melodic, meant to be heard for hours without fatigue, not an epic/cinematic score
- Reference the attached gameplay clips for exact pacing, animation timing, and layout — describe in text what's hard to catch on camera (e.g. hit-flash duration, drawer slide speed)
- (Optional twist, see Section 9) — floor visuals shift by biome tier every ~10 floors, reusing the same tileset family with palette/prop swaps rather than fully new art per tier

3. Heroes (the player's units)

- Tap to summon a hero, up to a concurrent cap

- A hero stays active until killed by the monster/floor — once killed, that summon slot frees up and can be resummoned by tapping again
- The **Summon** skill permanently raises the concurrent cap: $\text{Max Concurrent Heroes} = \text{Base Pool} + 2 \times \text{Summon Level}$ (max level 100)
- Heroes are the tap-driven force; **Soldiers** (Section 4) are a separate, auto-spawning force that doesn't require tapping

4. Soldiers

- Auto-spawn over time on a per-unit-type cooldown (spawn rate), independent of tapping
- Each soldier type (Swordsman, Archer, Ninja, Knight, Monk, etc.) has its own HP/Attack/Speed/Spawn-Rate curve — see the [Floor Scaling](#), [Wall Breakpoints](#), and research notes for the growth-rate pattern (~5.5-6.5%/level compounding, with sharp multiplier jumps at level walls like 100/1000/10000)
- Newer unit tiers start several times stronger than the previous tier AND scale faster per level — each new unit type is meant to eventually outpace the one before it, not just reskin it
- Speed/spawn-rate is a "personality" stat per unit (fast+fragile vs. slow+tanky), not tied to raw power — don't scale it off the same curve as HP/Attack

5. Floor / Difficulty Scaling

- $\text{Enemy HP} = \text{Base HP} \times (1 + \text{growth rate})^{(\text{floor} - 1)}$, growth rate around 8%/floor as a starting point (tune by feel)
- Every 10th floor is a "wall": HP multiplies by an extra ~3x on that floor, then normal growth resumes
- Full working model with editable inputs: [Floor Scaling](#) and [Wall Breakpoints](#) tabs in the spreadsheet

6. Gear / Items

- Gear isn't found at a random level — every item starts at level 1, and each time that same item drops again from a chest, its level goes up by 1 (repetition = progression, not replacement)
- Each item belongs to a **Family** (e.g. Claymore + Flamberge share a family; Tomahawk is its own family)
- **Same-family bonuses add together. Different families multiply onto each other.**
Formula: $\text{Final Stat} = \text{Base} \times (1 + \text{FamilyA total\%}) \times (1 + \text{FamilyB total\%}) \times \dots$
- Account-wide skills (like Power Up) multiply in last, on top of all gear families
- Full working model: [Gear Stacking Model](#) tab in the spreadsheet

- Drop rates cluster into a handful of rarity **pools** (one shared probability per pool, split evenly across the items in it) rather than each item having a hand-tuned individual rate — see **Loot Tiers** tab
- Items should have wildly different caps by design — some meant to max in your first session, some meant to take months, a few meant to be effectively unreachable trophies

7. Currency & Daily Rewards

Two separate currencies, with separate purposes — not one shared pool:

- **Gold** — earned from floor-clear kills. Spent on Hero & Skill upgrades and Soldier upgrades. This is the everyday, always-flowing currency.
- **Shop Currency** (daily-reward currency) — earned only from daily login (streak-based, increasing per consecutive day) and rewarded ads (capped number per day). Never directly purchasable. Spent exclusively in the Shop tab on:
 - A set of in-game boosts unique to the Shop
 - **The Fire Sword** — this is the *only* way to obtain the Fire Sword; it cannot drop from chests like other gear

Keeping these separate matters for the build: the Weapons tab (chest-drop gear, leveled by repetition per Section 6) and the Shop tab (daily-reward-currency purchases, including the one chest-exempt item) pull from two different currency sources and should not be interchangeable in the UI or the backend.

- Full model: **Gold Pacing (Floor Clears)** tab for Gold, **Shop Currency (Daily+Ads)** tab for Shop Currency — two separate tabs matching the two separate wallets

8. Skills

All 13 skills, their formulas (where known) and placeholders (where the source only said "a small percentage"), are in the **Hero Skills** tab. Categories to preserve in the build:

- **Core tap mechanic** (Heroes)
- **Passive capacity** (Summon)
- **Utility, not power** (Monster Seal, Monster Auto-Seal — these make re-climbing after a reset less tedious, they don't make you stronger)
- **Passive global multipliers** (Power Up)
- **Cross-stat dependency** (Fame — hero attack scales off total soldier attack, so investing in soldiers also strengthens the hero directly)
- **Temporary active buffs with cooldowns** (Doping, Rain of Gold, Warp, Ultimate Summon)

- **Prestige/reset** (New Dungeon — resets floor progress, keeps gear; reward curve in [Prestige Curve](#) tab)

9. Monetization

- 2-3 optional purchases only, none required to progress, no pay-to-win framing
- A rewarded-ad option ("watch ad for gold") placed off to the side, not front-and-center
- Persistent but unobtrusive top banner ad

10. Your Twist — Collector's Tower

Gear isn't just stat boosts — frame it with light TCG-style collection flavor (rarity tiers, a "collection log" viewable even for replaced items, maybe set-completion bonuses later). This is what differentiates the build from a straight clone — keep it in mind for naming, UI framing of the Weapons tab, and any future marketing.

11. Naming — Use IDs, Don't Lock In Real Names Yet

Every skill, item, and unit name in this brief and the spreadsheet is a placeholder pulled from the reference game for mechanical clarity. Do not reuse these names in the actual build. But you also don't need to invent final names right now — build the ID/display-name split from day one instead, and defer the creative naming decision indefinitely without it costing anything structurally later.

The pattern: every skill, item, and unit gets a permanent internal ID (never shown to the player, never changes) and a separate Display Name (freely editable, shown to the player only). All game logic — family matching, stacking math, save data — references the ID. Only the UI layer looks up the Display Name.

```
const ITEM_NAMES = {  
  
  "sword_t1": "Short Sword",  
  
  "sword_t2": "Longsword",  
  
  "axe_t1": "Battleaxe",  
  
  "armor_t1": "Light Armor",  
  
  "armor_t2": "Heavy Armor",  
  
  "helm_t1": "Great Helm",  

```

}

Rename later just means editing this one dictionary — nothing else in the codebase touches it.

The **Gear Stacking Model** spreadsheet tab already uses this pattern (Item ID / Family ID columns the formulas match on, separate Display Name / Family Display Name columns that are safe to edit freely) — mirror that same ID vs. display-name split in Godot for every system: items, skills, and the Soldiers/"Sellswords" roster alike.

When you do land on real names, a couple of starting directions worth keeping in mind:

- Gear families should read as a tiered lineage, not a flat list — e.g. a sword family might progress Training Sword → Short Sword → Longsword, an armor family Light Armor → Heavy Armor → Plate Armor, with each name change lining up with the family's per-item level tiers already modeled in the spreadsheet
- The Soldiers tab is a good candidate for its own themed rename (e.g. a mercenary/"Sellswords" framing with medieval unit names — Squire, Footman, Archer, Knight, Druid, Witch, Wizard) rather than keeping a generic "Soldiers" label
- Whatever the final names end up being, they only ever need to be written in the Display Name fields — the ID fields stay untouched

What to hand Replit, and in what order

1. This brief
2. **tower-climber-progression-model.xlsx** (all tabs — Floor Scaling, Wall Breakpoints, Gear Power Curve, Gold Pacing (Floor Clears), Shop Currency (Daily+Ads), Prestige Curve, Loot Tiers, Gear Stacking Model, Hero Skills)
3. The Godot starter project zip, with the instruction to extend it rather than start over
4. Gameplay reference clips, if you can capture them — title screen, one full floor clear, drawer opening, death/reset screen
5. Ask for one milestone at a time (e.g. "first, wire up the hero summon cap and resummon-on-death from Section 3") rather than "build the whole game" in one prompt — matches the staged next-steps order already in the starter kit's README